SemanticSync: Schema-Level Semantic Merge Policies for Local-First Relational Databases

Authors: Pathak Ashmit Akhilesh; Yash Bohra

Affiliation: VIT Vellore (department and exact institutional style to be confirmed)

Date: October 2026

Artifact repository: SemanticSync on GitHub

Abstract

Local-first software architectures let devices read and write local data while disconnected and synchronize later. CRDTs provide convergence for particular replicated data types under their stated assumptions; that does not mean every conflict-resolution rule preserves an application's intended meaning. For relational data, semantic interpretation, integrity constraints, and recovery behavior must be evaluated separately.

We describe SemanticSync, a prototype SQLite replication engine with three-way conflict detection, tracked operation replay, and schema-assigned pairwise resolution policies (`SUM`, `BOUNDED_SUM`, `AVG`, `APPEND`, `LWW`, `MIN`, `MAX`, and `REJECT`). The implementation also includes tested quarantine paths for dependent rows and checkpoint-assisted log compaction. These mechanisms do not constitute a general CRDT convergence proof or a complete relational replication guarantee.

The saved stress artifacts contain two paired trials for each of nine configurations, using the same generated trace within each pair and alternating execution order. The workload performs sequential tracked writes followed by batch synchronization. Across configurations, SemanticSync's conflict-only SCR ranges from 97.50% to 98.25% (the minimum and maximum configuration scores, not a confidence interval); LWW scores 0% under this workload's additive oracle. This result is specific to that oracle and does not establish general superiority. RVAR is 100% for benchmark-row values in the observed schedules, not full-database agreement. With only two trials, timing summaries are descriptive; paired deltas vary in sign and magnitude, so they support no general performance-advantage claim. The older single-run matrix is reported separately as historical evidence.

Keywords: local-first systems, SQLite replication, semantic merge, conflict resolution, referential integrity, checkpoint compaction

1. Introduction

1.1 The Local-First Relational Dilemma

Local-first systems let users work with local copies and synchronize later. Relational replication adds concerns beyond conflict selection: updates may express different business intent, schemas impose row and referential constraints, and replicas may reconnect with delayed or incomplete histories. CRDT research establishes convergence for specified data types under stated delivery and causal assumptions [11, 6]; it does not imply that every database replication system uses one model or that convergence alone establishes application-level correctness.

1.2 The Convergence vs. Correctness Paradox

Consider a warehouse management database with an `inventory` table containing a `quantity INTEGER` column. Suppose an item has an initial quantity of 100 in the common ancestor state:

- Replica A operates offline and sells 15 units (`UPDATE inventory SET quantity = 85 WHERE item_id = 1`).
- Replica B operates offline and sells 20 units (`UPDATE inventory SET quantity = 80 WHERE item_id = 1`).

Under a last-writer selection rule, if Replica B is selected as the winner:

The resolved quantity is 80.

Both replicas may store 80 after a successful write-back, while the additive interpretation of the two independent changes is 65. This illustrates a mismatch between a selected merge rule and the assumed business meaning; it is not a claim about all CRDTs or all LWW systems.

Similar failures occur across application domains:

- In medical record collaboration, averaging sensor observations (`AVG`) or appending multidisciplinary clinical notes (`APPEND`) is overwritten by whoever happened to submit last.

- In security-sensitive tables, concurrent modifications to a patient's blood type or an access control role are silently clobbered unless explicitly flagged for human inspection (`REJECT`).

- When a tracked parent delete arrives while dependent child rows are present, SemanticSync quarantines their payloads; a later child INSERT whose parent is absent is rejected for review. These outcomes preserve data/integrity for the tested paths, but do not automatically reconcile conflicting business intent.

1.3 Key Contributions

This artifact reports a prototype with schema-assigned, type-validated pairwise resolution policies; transactional quarantine on enumerated tracked parent-delete paths; and checkpoint-backed log pruning with ACK, membership, snapshot, history, and dependency checks. It also provides a synthetic stress harness for conflict-only SCR, benchmark-row RVAR, tracked-write timing, and batch-sync timing. These are implementation and evaluation contributions within the documented scope, not a claim of novelty over all related systems or a general convergence guarantee.

2. System Architecture

SemanticSync operates as a middleware layer positioned between local application SQL execution and the distributed replication network.

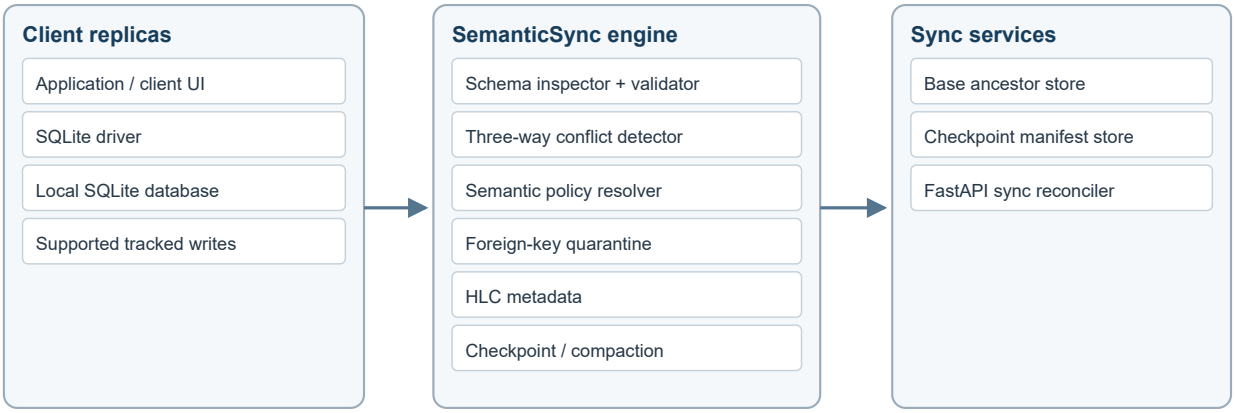


Figure 1. SemanticSync component layers and the flow from tracked replica writes through the engine to synchronization services.

2.1 Storage Layout and Tracked Operation Log

Each replica maintains a SQLite database file with metadata for tracked operations, quarantine, compacted receipts, and checkpoint markers. Only supported tracked mutation paths create operation records; arbitrary SQL and external writes are not captured. Eligible history may be removed after checkpoint validation, so the hot log is not permanently append-only:

- `_ss_change_log`: A hot log of column-level mutations (`table_name`, `row_id`, `column_name`, `old_value`, `new_value`, `ts_pt`, `ts_l`, `replica_id`, `op_type`, `origin_id`, `sequence`, `source_id`, `depends_on`, `bounded_delta`, `bounds_min`, `bounds_max`).

- `_ss_quarantine`: An isolated holding table storing orphaned child rows (`quarantine_id`, `table_name`, `row_id`, `row_payload`, `parent_table`, `parent_row_id`, `reason`, `quarantined_at`, `status`).

- `_ss_compacted_receipts`: Operation identities and payload hashes for pruned operations, retained as replay/duplicate evidence; this does not by itself establish complete causal history.

- `_ss_checkpoint_metadata`: Singleton tracking active checkpoint incarnation ID and snapshot epoch.

2.2 Hybrid Logical Clock Metadata

Hybrid logical clocks were introduced to combine physical-clock proximity with logical causality and consistent-snapshot reasoning [9]. SemanticSync stores HLC fields with tracked operations and uses supplied timestamps in selected comparisons, but this implementation does not establish a network-wide happened-before relation: synchronization is local and pairwise, and general row-mutation history does not encode complete causal dependencies. HLC values are not used alone to prove checkpoint stability.

3. Semantic Merge Policy Framework

3.1 Mathematical Formalization

Let V denote the domain of values for an attribute. A three-way merge takes a base value and the values and metadata from two replicas; it returns a resolved value or a flagged divergence that requires manual intervention.

For a pairwise merge to be independent of which replica is passed as the first argument, the resolver must treat each value together with its replica identity and metadata. This pairwise symmetry is useful, but is not by itself a convergence proof: associativity, duplicate handling, retained operation identity, and delivery semantics also matter. The equation below swaps complete replica inputs, not just bare values:

For the tested pairwise symmetry condition, swapping the complete replica inputs must preserve the resolver result: M(base, A, B) = M(base, B, A).

3.2 Supported Semantic Merge Policies

Table 1. Implemented policy behavior and properties demonstrated by tests.

Policy and accepted types	Implemented behavior	Tested properties and limits
`LWW` - any normalized type	Selects the greater supplied timestamp; ties use lexicographically greater replica ID. If either timestamp is missing, selects by replica ID.	Tests cover timestamp order, ID tie-break, missing-timestamp fallback, and swapping complete replica inputs. Determinism under these metadata rules is not causal ordering or semantic preservation. Same-ID/same-time conflicting payloads are unverified.
`SUM` - `INTEGER`, `REAL`, `NUMERIC`	Adds both replicas' changes relative to the common base value.	Generated integer tests cover commutativity and fixed-base grouping; repeat application is non-idempotent. Delta addition is associative over exact arithmetic for a fixed base. Floating-point rounding, changing bases, and arbitrary sequential histories are unverified.
`BOUNDED_SUM` - `INTEGER`, `REAL`, `NUMERIC`	Computes a fixed-base signed-delta candidate and checks inclusive bounds. Unsafe candidates are flagged/pending, not clamped or materialized.	Seeded integer tests swap operands and compare status and candidate; separate tests cover bounds and pending retries. Symmetry is for fixed operands and bounds. It is not a join-semilattice; arbitrary delivery order or mesh convergence is unverified.
`AVG` - `INTEGER`, `REAL`, `NUMERIC`	Returns the pairwise mean of the two current values; retains no sample count or weight.	Tests cover pairwise commutativity and a concrete non-associativity example. Accurate group means require counts or weights; multi-sample behavior is unverified.
`APPEND` - `TEXT`	Accepts the changed side if the other equals base; equal current strings return once; otherwise joins full values with `;` in replica-ID order.	Tests cover pair symmetry with IDs, identical inputs, and one repeat-sync schedule. Contribution identity/deduplication, general associativity, N-way idempotence, and multi-author convergence are unverified.
`MIN` / `MAX` - `INTEGER`, `REAL`, `NUMERIC`	Returns the numeric minimum or maximum.	Generated integer tests cover commutativity, associativity, and idempotence. NaN and arbitrary SQLite coercion cases are unverified.
`REJECT` - any normalized type	Flags a conflict and returns no value for automatic write-back.	Tests cover input-order invariant flagged results. Later manual resolution and its multi-replica behavior are separate and not implied.

These tests exercise the resolver or selected sync paths; they do not establish general convergence across arbitrary schedules. Mathematical laws apply only to the stated domains and representations. For example, SUM's fixed-base delta formula is associative over exact arithmetic, while ordinary floating-point operations can round differently; APPEND's pairwise ordering does not create a retained set of authors or contributions.

3.3 Tombstone Reconciliation (Update vs. Delete)

When the detector reports an update/delete row conflict, the pairwise sync path supports table-level choices. `FLAG_FOR_REVIEW` is the default and records a conflict without automatic write-back. `DELETE_WINS` attempts a paired delete; a write failure leaves the conflict flagged. `UPDATE_RESURRECTS` requires a dictionary row payload and attempts a paired upsert; absent or malformed payloads remain unresolved. These choices do not classify general causal ordering, and tests cover selected two-replica histories rather than arbitrary delivery schedules.

3.4 Type Compatibility and Fail-Fast Validation

Before runtime execution, policies assigned via `PUT /api/v1/databases/{id}/policies` are validated against normalized SQLite column types. `SUM`, `AVG`, `MIN`, `MAX`, and `BOUNDED_SUM` accept `INTEGER`, `REAL`, or `NUMERIC`; `APPEND` accepts `TEXT`; `LWW` and `REJECT` use the registry wildcard. Incompatible types fail validation. The resolver's Python comparisons may accept more values in isolation than the policy validator permits for a configured column.

4. Foreign-Key Quarantine Boundary

In relational databases, foreign-key constraints define referential integrity invariants. In SQLite, this is enforced via `PRAGMA foreign_keys = ON`.

4.1 The Referential Cascade Problem

Suppose Replica A inserts an `order_items` record referencing `order_id = 10` while Replica B deletes `orders` record `10`. If the insert arrives after the parent is gone, FK enforcement rejects the insert and reports it as a rejected operation. If a tracked delete is applied while dependent rows are present, the supported delete path quarantines those rows before removing the parent. Quarantine preserves payloads but does not automatically reconcile the conflicting business intent; an operator may need to review the insert or quarantined child.

4.2 Quarantine Isolation Protocol

For the supported tracked delete paths, SemanticSync uses a transactional quarantine boundary:

1. **Connection Enforcement:** Tracked local writes, operation delivery, paired conflict resolution, and quarantine recovery open SQLite connections with `PRAGMA foreign_keys = ON` before beginning their write transaction.

2. **Delete Handling:** Local deletes, received `DELETE` and `RESOLVE`-delete operations, and paired engine/API delete resolutions recursively identify declared dependent rows and serialize their payloads into `_ss_quarantine` before deleting the parent.

3. **Atomic Validation:** Row removal, quarantine records, and the tracked operation/receipt share a transaction. `PRAGMA foreign_key_check` is run before commit; an error rolls the transaction back.

4. **Parent-Key Updates:** These are not quarantined. SQLite's declared `ON UPDATE` action applies (for example, `CASCADE`) or the tracked transaction is rejected when the constraint blocks the update.

5. **Operator Recovery:** Quarantined rows can be relinked to an existing parent or marked `confirmed_deleted` through transactional API handling, subject to the supported single-column parent-key and single-column-FK relink limits.

These guarantees cover the tested tracked paths, not arbitrary database access. Direct SQL-console writes use FK-enabled connections but do not create operation identities or quarantine dependent rows, and do not use the shared mutation lock. SQLite's declared FK action or constraint rejection governs those writes. External processes can open connections with foreign keys disabled.

5. Checkpoint-Assisted Log Compaction

Uncompacted tracked-operation logs can grow over time and add replay cost; this implementation prunes only under the checkpoint conditions described below.

5.1 Compaction Eligibility and Recovery Checks

Compacting a change log prematurely can discard history needed by a lagging replica. In the implemented `/sync/compact` path, an operation's presence in every enrolled replica's durable ACK vector is necessary evidence, but not sufficient authorization. The checks below describe the tested tracked-operation path, not a formal proof for distributed execution.

5.2 Compaction Protocol

1. **ACK and horizon evidence:** Recovery requires the committed manifest's member IDs to equal the complete currently enrolled replica set, and its ACK-vector keys to match those members. Candidate IDs must be present in every vector with the same non-null payload hash. The stable cutoff is the minimum, over replicas, of each replica's maximum HLC among acknowledged operations; each local log entry must be strictly below it. HLC is a cutoff after exact-ID evidence, never proof of delivery. Missing vectors or an empty common set retain the logs. A membership change after manifest commitment blocks that checkpoint before ACK reconciliation or pruning.

2. **Committed checkpoint evidence:** The explicitly initialized checkpoint store, store identity, committed manifest, full archived payload hashes, membership/incarnation/path data, and one-to-one snapshot set must validate. Snapshot file hashes, SQLite integrity, embedded identity, schema/data fingerprints, and exact operation/receipt frontier are recomputed. The live replica must match the snapshot plus a replay-validated tracked tail, and every acknowledged operation must be represented consistently in replica history.

3. **Durable ACK reconciliation:** Before pruning, recovery validates every member's ACK vector and registry evidence. ACK gaps from a committed manifest may be reconciled only after all members pass preflight; persistence is checked and failed in-memory updates roll back.

4. **Per-entry eligibility:** In each replica transaction, an entry must have a unique operation identity, match the acknowledged payload hash, and have dependencies either included in the eligible set or already represented by a compacted receipt. Entries at or above the strict HLC cutoff, duplicate identities, hash mismatches, and entries with unresolved dependencies remain in the hot log.

5. **Atomic local prune:** An eligible operation's compacted receipt and checkpoint marker are written atomically with its hot-log deletion in that replica's `BEGIN IMMEDIATE` transaction. The checkpoint journal advances through `PRUNING` to `COMPLETE` only after all members succeed. A committed partial checkpoint can be retried idempotently; invalid or ambiguous evidence blocks the affected checkpoint without authorizing its pruning.

6. **Trust boundary and limitations:** `compact_change_log()` is a low-level helper and trusts its caller-supplied eligible-ID set and cutoff; it does not independently verify the registry or checkpoint manifest. The production API builds those arguments only after the checks above. Direct Python callers are trusted internal code, not an independently safe authorization boundary. Coordination uses one process-wide lock; multiple processes and power-loss durability are not certified.

6. Empirical Evaluation

6.1 Experimental Setup

The seeded stress harness creates private temporary SQLite databases. Each paired trial uses the same generated operation list, equivalent base rows and replica initialization, and one execution of each system; system order alternates by trial. It records Python version, platform, machine and processor strings returned by the runtime, SQLite runtime version, journal mode and FK setting, seed, workload parameters, trace digest, trial index/order, and policies. It does not infer storage-device identity or control system load. Writes run sequentially before batch sync, not concurrently.

The code contains two distinct evaluation paths:

1. **Domain scenario runner:** Three illustrative SQLite fixture workloads scored by the API runner; this path does not execute replica synchronization and its numerical results are not retained as verified measurements (Section 6.3).

2. **Three-by-three parametric stress harness:** Scaled synthetic workloads across 1,000, 5,000, and 10,000 operations evaluated at 5%, 20%, and 50% conflict densities (seed 42, configurable paired trials; two trials by default).

6.2 Evaluation Metrics

- **Semantic Correctness Rate (SCR):** In the stress harness, the proportion of final values for generated conflict rows that match the independently constructed semantic oracle. Conflict rows are identified by the workload's `is_conflict` flag, carried with each operation ID and written to the artifact as `conflict_row_ids`. Both replicas' final values are scored. Non-conflict rows are excluded; the denominator is twice the number of conflict rows. A zero-conflict run has no SCR denominator and reports N/A (`null` in JSON and blank in CSV).

- **Benchmark-Row Value Agreement Rate (RVAR):** The proportion of ordered `bench_rows.value` pairs that are equal across the two replicas. It excludes schemas, internal logs, receipts, quarantine records, and all other database state; it is not full-database convergence. A passing RVAR does not establish equal database contents.

- **Valid Resolution Accuracy (VRA):** Percentage of conflict resolutions that satisfy domain range and referential integrity constraints. The synthetic stress workload defines neither such constraints nor a validity oracle, so stress VRA is reported as N/A.

- **Stress Batch Sync Delta:** SemanticSync batch sync time minus LWW batch sync time. It includes the measured batch synchronization pipeline and is not isolated policy-execution time.

- **Per-Operation Mutation Latency Percentiles (p50, p95, p99):** Wall-clock time around each replica-local `apply_change` call, including SQLite mutation and change-log capture. Writes are sequential, not simultaneous. Reported summaries include sample count, median, minimum, and maximum across trials, in milliseconds.

- **Batch Synchronization Duration:** Wall-clock time around `sync_replicas` only; excludes database creation and tracked writes. Each system's durations and the paired SemanticSync-minus-LWW delta are recorded by trial and summarized in milliseconds. This is a system-level sync timing, not resolver-only overhead.

6.3 Domain Scenario Runner (Not a Reproducible Result Set)

The inventory, notes, and healthcare workload classes create SQLite fixtures and list expected conflict scenarios. The former API benchmark implementation scored the replica-B value as an LWW proxy, called `SemanticResolver.resolve` directly without executing the replica synchronization engine, and emitted hard-coded convergence values. The current `/benchmark/run` endpoint returns `evaluation_status="illustrative_unvalidated"` with comparative metric fields set to null. Its output is not empirical evidence of comparative performance or convergence. Checked-in tests verify this status response and fixture setup, not domain scenario scores; no validated per-run domain result artifact is saved. The former domain SCR percentages, evaluation-loop timings, and convergence figures are excluded from empirical results. A valid domain experiment needs an independent oracle, actual policy and baseline execution, saved run records, and assertions over outcomes.

6.4 Historical Single-Trial Stress Matrix (2026-10-13)

The following values are historical single-run measurements from `artifacts/benchmarks/historical-matrix-optimized-20261013/` (seed 42, one run per configuration). They are not the repeated-trial results or estimates. The files were regenerated under the corrected metric definitions and should be treated only as single-run observations:

Table 2. Historical single-run correctness observations (2026-10-13).

Operations	Density	Conflict rows	LWW / SemanticSync SCR	LWW / SemanticSync RVAR	VRA
1,000	5%	50	0.00% / **98.00%**	100.00% / 100.00%	N/A
1,000	20%	200	0.00% / **97.50%**	100.00% / 100.00%	N/A
1,000	50%	500	0.00% / **98.20%**	100.00% / 100.00%	N/A
5,000	5%	250	0.00% / **98.00%**	100.00% / 100.00%	N/A
5,000	20%	1,000	0.00% / **98.00%**	100.00% / 100.00%	N/A
5,000	50%	2,500	0.00% / **98.16%**	100.00% / 100.00%	N/A
10,000	5%	500	0.00% / **98.20%**	100.00% / 100.00%	N/A
10,000	20%	2,000	0.00% / **98.25%**	100.00% / 100.00%	N/A
10,000	50%	5,000	0.00% / **97.70%**	100.00% / 100.00%	N/A

Table 3. Historical single-run timing observations (2026-10-13).

Operations	Density	Mutation p50 / p95 / p99 (ms)	LWW / SemanticSync sync (s)	Sync delta (s)
1,000	5%	3.34 ms / 3.75 ms / 4.04 ms	1.010 s / 1.033 s	0.023 s
1,000	20%	3.31 ms / 3.76 ms / 4.93 ms	2.957 s / 3.254 s	0.297 s
1,000	50%	3.31 ms / 3.75 ms / 4.07 ms	6.919 s / 7.096 s	0.177 s
5,000	5%	3.45 ms / 3.88 ms / 4.35 ms	10.669 s / 10.740 s	0.071 s
5,000	20%	3.45 ms / 3.93 ms / 4.43 ms	23.582 s / 23.700 s	0.118 s
5,000	50%	3.46 ms / 3.94 ms / 4.51 ms	50.143 s / 50.056 s	-0.087 s

Operations	Density	Mutation p50 / p95 / p99 (ms)	LWW / SemanticSync sync (s)	Sync delta (s)
10,000	5%	3.58 ms / 4.15 ms / 4.64 ms	36.156 s / 35.727 s	-0.429 s
10,000	20%	3.59 ms / 4.13 ms / 4.63 ms	68.028 s / 68.166 s	0.138 s
10,000	50%	3.60 ms / 4.20 ms / 4.67 ms	138.337 s / 139.082 s	0.745 s

SCR is scored only on conflict rows and both replicas' final values; RVAR compares only `bench_rows.value` pairs. Stress VRA is N/A because this workload defines no range or referential-integrity constraints. The table reports saved single-run values; seconds are converted from recorded millisecond durations. Negative deltas mean SemanticSync took less time in that particular run. These environment-sensitive observations are not isolated policy costs, repeated-trial evidence, or a general performance guarantee. Earlier matrix values used an all-row correctness denominator and are stale for these metric definitions. The original `artifacts/benchmarks/historical-matrix-20261013/` remains unchanged; the optimized output files were regenerated in place, and their previous committed versions remain in Git history.

6.5 Updated paired stress measurements (complete three-by-three matrix)

The saved `artifacts/benchmarks/repeated-stress-20261014/` artifacts contain all nine configurations with two paired trials per configuration. Results below are descriptive medians; ranges show the minimum and maximum of the two trials. These measurements are separate from the historical single-run table above and are not evidence of statistical significance.

Table 4. Repeated-trial correctness and benchmark-row agreement observations.

Operations	Density	LWW / SemanticSync SCR (%)	LWW / SemanticSync RVAR (%)
1,000	5%	0.0 / 98.0	100 / 100
1,000	20%	0.0 / 97.5	100 / 100
1,000	50%	0.0 / 98.2	100 / 100
5,000	5%	0.0 / 98.0	100 / 100
5,000	20%	0.0 / 98.0	100 / 100
5,000	50%	0.0 / 98.2	100 / 100
10,000	5%	0.0 / 98.2	100 / 100
10,000	20%	0.0 / 98.2	100 / 100
10,000	50%	0.0 / 97.7	100 / 100

Table 5. Repeated-trial batch-sync timing medians and ranges.

Operations	Density	LWW sync median [range] (s)	SemanticSync sync median [range] (s)	Paired delta median [range] (ms)
1,000	5%	1.109 [1.091-1.127]	1.132 [1.120-1.143]	22.79 [16.31-29.26]
1,000	20%	3.435 [3.333-3.537]	3.363 [3.335-3.392]	-71.81 [-145.17-1.55]
1,000	50%	7.881 [7.847-7.915]	7.774 [7.766-7.782]	-107.20 [-149.34--65.06]
5,000	5%	11.345 [11.335-11.354]	11.340 [11.228-11.451]	-5.02 [-107.00-96.97]
5,000	20%	29.214 [25.044-33.384]	32.019 [30.885-33.152]	2,804.71 [-231.96-5,841.38]
5,000	50%	74.180 [70.128-78.231]	75.602 [72.763-78.440]	1,421.64 [208.90-2,634.38]
10,000	5%	76.220 [73.793-78.646]	74.825 [73.058-76.593]	-1,394.30 [-2,053.43--735.17]
10,000	20%	118.678 [98.276-139.080]	147.645 [143.627-151.664]	28,967.53 [4,546.73-53,388.32]
10,000	50%	193.364 [192.004-194.723]	188.088 [184.699-191.476]	-5,276.10 [-10,024.15--528.04]

Each paired trial executes the same seeded operations against equivalent initial rows, with execution order alternated. JSON and CSV records include per-trial configuration, workload digest, metadata available from the runtime, operations, observations, and latency samples; see `docs/TESTING_STRATEGY.md`. The two-trial sample is small and system load is uncontrolled. SCR uses

only conflict rows and the additive oracle; RVAR compares only `bench_rows.value`. The schedule remains sequential local writes followed by batch sync. The 5,000-operation/20% cell has substantial spread: trial deltas were +5,841.38 ms and -231.96 ms. The 10,000-operation/20% cell also varies (+4,546.73 ms and +53,388.32 ms). Both cells use the same within-pair digest, operation count, initial-state specification, timing boundaries, and alternating order; with two trials, the source of variability cannot be isolated, so neither trial is excluded.

6.6 Key Findings

1. **Historical conflict-row semantic accuracy:** For this generated additive SUM oracle, LWW scores 0% because the selected single-replica value omits one of the two deltas. This workload-specific result does not establish general SemanticSync superiority. SemanticSync results in this single-trial matrix are historical and must not be presented as repeated-trial estimates. RVAR compares benchmark-table values only.

2. **Timing:** These sequential runs do not establish contention behavior, statistical significance, or general performance scaling. The two-trial ranges, especially at 5,000 operations/20% density and 10,000 operations/20% density, show that timing is variable on this host.

3. **Stress Timing Scope:** The matrix reports tracked-write measurements and batch-sync deltas; batch synchronization is not isolated policy-execution cost. The former domain loop timing is unvalidated and excluded from empirical results.

7. Related Work and Scope of Comparison

CRDT theory defines convergence conditions for specified replicated types; the framework and machine-checked examples in Shapiro et al. [11] and Gomes et al. [6] are formal results for their models, not evidence about SemanticSync. Kleppmann and Beresford [7] give a formal JSON CRDT, and Kleppmann et al. [8] describe local-first design principles. SemanticSync is an implementation artifact with selected pairwise policies and schedules; it is not established as a CRDT by this work.

Work on invariants under weak consistency provides important prior approaches. Balegas et al. [1] develop an escrow-inspired bounded counter; Balegas et al. [2] present static analysis and operation changes for invariant preservation; Lewchenko et al. [10] introduce consistency guards for conflict-aware replicated data types. SemanticSync's `BOUNDED_SUM` is a bounded tracked signed-delta path with pending rejection, not an escrow allocation protocol or a formal invariant-preserving analysis. Similarity of goals does not establish novelty or equivalence.

Edwards, Petricek, and van der Storm [5] present challenge problems concerning schema change in live programming and local-first software. That work is not evidence for claims about foreign-key orphan handling. The `cr-sqlite` project and ElectricSQL describe different replication architectures [12, 13]; this manuscript does not claim an exhaustive feature comparison. Spanner and Dynamo [3, 4] are examples of distributed database designs with different consistency and availability trade-offs, not systems that can be dismissed as incompatible with local-first applications on the basis of this prototype.

This related-work discussion is selective, not a systematic literature review. The repository does not establish a novelty claim for schema-level policies, quarantine, or checkpointing. A submission would need a broader, current comparison against relational sync systems, invariant-preserving replication, conflict-resolution frameworks, and application-specific merge systems.

8. Discussion and Known Limitations

1. **Policy algebra:** Resolver tests cover finite generated inputs and concrete examples. `SUM`'s arithmetic laws apply to fixed-base deltas under exact arithmetic, not arbitrary repeated merge histories. `AVG` is pairwise and non-associative. `APPEND` retains no contribution identity. LWW's tie rule is deterministic for supplied metadata, not causal. No policy set has a general arbitrary-schedule convergence proof.

2. **Operation propagation:** Synchronization is explicit and pairwise. It preserves or rejects selected operations based on expected row state and stable identity; local divergence is surfaced for review. No network transport or general offline-history merge is implemented.

3. **Relational constraints:** Quarantine applies only to enumerated tracked delete paths and supported key/reference shapes. Composite parent keys and `WITHOUT ROWID` dependent rows are rejected. Parent-key updates follow declared SQLite actions, not quarantine. SQL-console and external-process writes are not tracked or quarantined; external connections may disable FK enforcement. Multi-process coordination is unsupported.

4. **Checkpoint recovery:** Production pruning requires exact ACK evidence and payload hashes, complete current membership, committed validated manifest/archive/snapshots, validated replica histories and tails, durable ACK reconciliation, dependency

closure, strict HLC cutoff, and per-replica transactions. The low-level compaction helper trusts its caller-supplied eligibility set and cutoff. RESOLVE tails require manual review. Coordination uses a single-process lock; power-loss durability is not certified.

5. **Benchmark scope:** The repeated stress matrix has two paired trials per configuration, seed 42, and alternated execution order. It uses sequential tracked writes followed by batch synchronization. SCR is conflict-only against an additive oracle; the 0% LWW result follows from this workload's oracle and does not establish general SemanticSync superiority. RVAR compares only `bench_rows.value`; VRA is unmeasured. Timing ranges are wide in some configurations, system load is uncontrolled, and results do not establish statistical significance or general performance advantage. The current domain API returns `illustrative_unvalidated` with comparative metrics null. The former resolver-only proxy and hard-coded convergence outputs are excluded from empirical results.

6. **Phase evidence:** Phase 6 is approved for its documented tested scope. The latest user-reported Windows/Python 3.14 results are 41 focused checkpoint tests passed (3.30 s) and 309 backend tests passed with one warning (75.06 s); these were not rerun in this audit. This evidence does not certify power-loss durability, multi-process coordination, every crash boundary, or arbitrary-schedule convergence.

9. Conclusion

This artifact implements selected schema-level merge policies, tracked operation handling, relational quarantine paths, and checkpoint-backed log pruning for SQLite replicas. The tests and benchmark artifacts provide evidence only for their enumerated policies, schemas, inputs, and schedules. The stress matrix reports conflict-only correctness and benchmark-row agreement for a small, sequential workload; it does not demonstrate full-database convergence, concurrent-writer performance, statistical significance, or a general performance advantage. Broader correctness and comparative claims require formal specifications, wider schedule testing, reproducible domain runs, and a systematic literature comparison.

References

1. Balegas, V., Serra, D., Duarte, S., Ferreira, C., Rodrigues, R., Preguiça, N., Shapiro, M., and Najafzadeh, M. (2015). “Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants.” arXiv:1503.09052.

2. Balegas, V., Duarte, S., Ferreira, C., Rodrigues, R., and Preguiça, N. (2018). “IPA: Invariant-Preserving Applications for Weakly-Consistent Replicated Databases.” *Proceedings of the VLDB Endowment*, 12(4), 404-418. DOI: 10.14778/3297753.3297760.

3. Corbett, J. C., et al. (2013). “Spanner: Google’s Globally-Distributed Database.” *ACM Transactions on Computer Systems*, 31(3), Article 8. DOI: 10.1145/2491245.

4. DeCandia, G., et al. (2007). “Dynamo: Amazon’s Highly Available Key-value Store.” In *Proceedings of SOSP 2007*, 205-220. DOI: 10.1145/1294261.1294281.

5. Edwards, J., Petricek, T., and van der Storm, T. (2023). “Live & Local Schema Change: Challenge Problems.” arXiv:2309.11406.

6. Gomes, V. B. F., Kleppmann, M., Mulligan, D. P., and Beresford, A. R. (2017). “Verifying Strong Eventual Consistency in Distributed Systems.” *Proceedings of the ACM on Programming Languages*, 1(OOPSLA), Article 109. DOI: 10.1145/3133933.

7. Kleppmann, M., and Beresford, A. R. (2017). “A Conflict-Free Replicated JSON Datatype.” *IEEE Transactions on Parallel and Distributed Systems*, 28(10), 2733-2746. DOI: 10.1109/TPDS.2017.2697382.

8. Kleppmann, M., Wiggins, A., van Hardenberg, P., and McGranaghan, M. (2019). “Local-First Software: You Own Your Data, in Spite of the Cloud.” *Onward! 2019*, 154-178. DOI: 10.1145/3359591.3359737.

9. Kulkarni, S. S., Demirbas, M., Madappa, D., Avva, B., and Leone, M. (2014). “Logical Physical Clocks.” In *18th International Conference on Principles of Distributed Systems (OPODIS 2014)*, *Lecture Notes in Computer Science*, vol. 8878, pp. 17-32. DOI: 10.1007/978-3-319-14472-6_2. An earlier University at Buffalo CSE technical report (2014-04) has the expanded title “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases.”

10. Lewchenko, N. V., Radhakrishna, A., Gaonkar, A., and Černý, P. (2018). “Conflict-Aware Replicated Data Types.” arXiv:1802.08733.

11. Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). “Conflict-Free Replicated Data Types.” In *13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, 386-400. DOI: 10.1007/978-3-642-24550-3_29.

12. The `cr-sqlite` project. Official source repository (accessed 2026-10-15); software reference, not a peer-reviewed publication.

13. ElectricSQL project. Official source repository (accessed 2026-10-15); software reference, not a peer-reviewed publication.